

The Coward King

Database Design Document

Tecnológico de Monterrey

José Abel Domínguez Rish

Leonardo Flores Mendoza

Nicolás Casillas Larrañaga

1. Project Overview

The Coward King is a turn-based roguelite game where players defend their king by deploying cards against incoming enemy hordes. Each run is independent, and players accumulate permanent upgrades across defeats. The database supports three core systems: run progression (hordes, levels, gold), the card economy (base stats and per-player upgrades), and player analytics (stats, leaderboard, and admin balancing views).

2. Entity-Relationship Schema

Ten main entities are identified. The following sections describe each entity, its attributes, and its relationships to others.

2.1 PLAYER

Represents each registered user. Stores credentials and creation date. It is the root entity: every run, upgrade, and statistic belongs to a player. Enables the login/signup system.

Attribute	Domain / Type	Constraint	Description
player_id	INT	PK / NOT NULL	Unique player identifier
username	VARCHAR(50)	NOT NULL / UNIQUE	Display name shown on screen
password_hash	VARCHAR(64)	NULLABLE	Hashed password for authentication
created_at	TIMESTAMP	DEFAULT NOW()	Player registration date

2.2 RUN

Records each attempt by a player. Captures the current level and horde, accumulated gold, and the final result. Essential for the checkpoint system and the Game Over screen.

Attribute	Domain / Type	Constraint	Description
run_id	INT	PK / NOT NULL	Unique run identifier
player_id	INT	FK → PLAYER	Owner player
current_level_id	INT	FK → LEVEL	Current level of the run
total_gold_earned	INT	DEFAULT 0	Total gold earned in the run
current_horde	INT	DEFAULT 1	Current horde of the run
started_at	TIMESTAMP	NOT NULL	Run start time

ended_at	TIMESTAMP	NULLABLE	Run end time (NULL if in progress)
result	ENUM	NOT NULL	'victory', 'defeat', or 'in_progress'
is_active	BOOL	DEFAULT TRUE	Indicates if the run is ongoing

2.3 LEVEL

Static catalog table. Defines the three game levels (Catacombs, Dungeon, Castle) with their boss, visual theme, and number of hordes. Centralizing this avoids hardcoding it in application logic and facilitates future balancing.

Attribute	Domain / Type	Constraint	Description
level_id	INT	PK / NOT NULL	Unique level identifier
level_number	INT	NOT NULL	Order of the level in the game
name	VARCHAR(50)	NOT NULL	Level name
theme	VARCHAR(50)	NOT NULL	Visual theme or environment
total_hordes	INT	DEFAULT 3	Total number of hordes in the level
boss_name	VARCHAR(50)	NOT NULL	Name of the level's final boss (justified exception to 3NF — see Section 4)

2.4 HORDE

Defines the hordes within each level. Stores balance parameters such as maximum turns and enemies per spawn. Separating HORDE from LEVEL allows each horde to have independent configuration, which is useful if difficulty per horde within the same level needs to vary in the future.

Attribute	Domain / Type	Constraint	Description
horde_id	INT	PK / NOT NULL	Unique horde identifier
level_id	INT	FK → LEVEL	Level the horde belongs to
horde_number	INT	NOT NULL	Horde number within the level
max_turns	INT	DEFAULT 30	Maximum allowed turns

enemies_per_spawn	INT	DEFAULT 3	Enemies generated per spawn
-------------------	-----	-----------	-----------------------------

2.5 RUN_HORDE

Association table between a run and the hordes played. Records concrete progress: turns survived and enemies eliminated per horde. It is the data source for the statistics screen and for validating the checkpoint upon death.

Attribute	Domain / Type	Constraint	Description
run_horde_id	INT	PK / NOT NULL	Unique relationship identifier
run_id	INT	FK → RUN	Run this entry belongs to
horde_id	INT	FK → HORDE	Horde played
turns_survived	INT	DEFAULT 0	Number of turns survived
enemies_killed	INT	DEFAULT 0	Enemies eliminated in the horde
completed	BOOL	DEFAULT FALSE	Whether the horde was completed

2.6 CARD

Static catalog of the 15 cards in the game. Stores the base stats of each card (AP cost, HP, damage) and its type. This table is the reference point for the upgrade system: upgrades are deltas over the base values stored here.

Attribute	Domain / Type	Constraint	Description
card_id	INT	PK / NOT NULL	Unique card identifier
name	VARCHAR(50)	NOT NULL	Card name
type	ENUM	NOT NULL	'attack', 'defense', or 'political'
base_ap_cost	INT	NOT NULL	Base action point cost
base_hp	INT	DEFAULT 0	Base hit points
base_damage	INT	DEFAULT 0	Base damage
range_attack	INT	DEFAULT 1	Attack range in tiles
duration	INT	DEFAULT 0	Effect duration in turns (0 = permanent)
effect_description	TEXT	NULLABLE	Human-readable effect description

2.7 CARD_UPGRADE

Records permanent upgrades a player has purchased for each card. Upgrades survive deaths and accumulate across runs. Modeling this as a separate table (instead of columns in CARD) allows a complete history of gold spending per player.

Attribute	Domain / Type	Constraint	Description
upgrade_id	INT	PK / NOT NULL	Unique upgrade identifier
player_id	INT	FK → PLAYER	Owner player
card_id	INT	FK → CARD	Upgraded card
level	INT	DEFAULT 1	Current upgrade tier (1–3)
ap_cost_bonus	INT	DEFAULT 0	AP cost modifier (negative = cheaper)
hp_bonus	INT	DEFAULT 0	Hit point bonus
damage_bonus	INT	DEFAULT 0	Damage bonus
gold_spent	INT	DEFAULT 0	Gold invested in this upgrade

2.8 RUN_HAND

Records which cards form the player's hand in each horde of a run. Allows reconstructing the hand of any horde for analysis. The is_retained field indicates whether the card was kept during the horde transition (the 1-card retention mechanic).

Attribute	Domain / Type	Constraint	Description
hand_id	INT	PK / NOT NULL	Unique entry identifier
run_id	INT	FK → RUN	Run this hand belongs to
horde_id	INT	FK → HORDE	Horde where the hand was used
card_id	INT	FK → CARD	Card included in the hand
is_retained	BOOL	DEFAULT FALSE	Whether the card was retained across this horde

2.9 ENEMY

Static catalog of enemy types by level (Skeleton, Ogre, Elite Warrior and their boss versions). Stores base stats and the is_boss flag. Being linked to LEVEL allows knowing which enemies appear in each level without hardcoding the logic in the application.

Attribute	Domain / Type	Constraint	Description
enemy_id	INT	PK / NOT NULL	Unique enemy identifier
level_id	INT	FK → LEVEL	Level the enemy belongs to
name	VARCHAR(50)	NOT NULL	Enemy name
type	ENUM	NOT NULL	'normal' or 'boss'
hp	INT	NOT NULL	Enemy hit points
damage	INT	NOT NULL	Damage dealt by the enemy
speed	INT	DEFAULT 1	Movement speed in tiles per turn
is_boss	BOOL	DEFAULT FALSE	Differentiates boss vs. normal unit

2.10 STATS

Aggregates global player statistics across all runs (not per individual run). Allows displaying the Stats screen in the main menu. This is a single-row table per player (1:1 relationship with PLAYER), auto-created by trigger on player registration.

Attribute	Domain / Type	Constraint	Description
stat_id	INT	PK / NOT NULL	Unique record identifier
player_id	INT	FK → PLAYER / UNIQUE	Owner player
total_runs	INT	DEFAULT 0	Total runs started
total_enemies_killed	INT	DEFAULT 0	Total enemies eliminated
total_upgrades	INT	DEFAULT 0	Total upgrades performed
total_gold_earned	INT	DEFAULT 0	Total gold earned historically
levels_completed	INT	DEFAULT 0	Total levels completed

3. Relationships, Cardinalities, and Constraints

Relationship	Cardinality	Referential Integrity
PLAYER → RUN	1 player : N runs	ON DELETE CASCADE: deleting a player removes all their runs.
PLAYER → CARD_UPGRADE	1 player : N upgrades	ON DELETE CASCADE: deleting a player removes all their upgrades.
PLAYER → STATS	1 player : 1 stats record	ON DELETE CASCADE; player_id UNIQUE in STATS.
RUN → LEVEL	N runs : 1 level	ON DELETE RESTRICT: a level cannot be deleted while runs reference it.
RUN → RUN_HORDE	1 run : N horde records	ON DELETE CASCADE.
RUN → RUN_HAND	1 run : N hand entries	ON DELETE CASCADE.
LEVEL → HORDE	1 level : N hordes	ON DELETE CASCADE; UNIQUE(level_id, horde_number).
LEVEL → ENEMY	1 level : N enemy types	ON DELETE CASCADE.
HORDE → RUN_HORDE	1 horde template : N run records	ON DELETE RESTRICT.
HORDE → RUN_HAND	1 horde : N hand entries across runs	ON DELETE RESTRICT.
CARD → RUN_HAND	1 card : N hand entries across runs	ON DELETE RESTRICT.
CARD → CARD_UPGRADE	1 card : N upgrades across players	ON DELETE RESTRICT.

3.1 Additional Integrity Constraints

Constraint Type	Applies To	Rule / Description
CHECK (non-negative)	total_gold_earned, gold_spent, turns_survived, enemies_killed, hp, damage, all stats	Numeric values must be >= 0.
CHECK (valid range)	current_horde, horde_number, level_number	Values must stay within valid game-design ranges (e.g., level_number 1–3).
UNIQUE	CARD_UPGRADE (player_id, card_id)	Each player may have only one upgrade record per card.

UNIQUE	RUN_HORDE (run_id, horde_id)	The same horde cannot be recorded twice within a single run.
UNIQUE	HORDE (level_id, horde_number)	Within a level, horde numbers must be unique.
CHECK (valid result)	RUN.result	Only accepts 'victory', 'defeat', or 'in_progress'.
CHECK (date order)	RUN (started_at, ended_at)	ended_at must be NULL or greater than started_at.

4. Normalization Justification

4.1 Importance of Normalization

Database normalization is the process of organizing a relational database to reduce data redundancy and improve data integrity. By following normal forms, each piece of information is stored in exactly one place, which makes the database easier to maintain, update, and scale. For The Coward King, normalization was especially important because the game involves multiple interrelated entities: players, runs, cards, enemies, hordes, and upgrades. Without normalization, changes to a card's stats would require updates in multiple places and could cause inconsistencies that produce bugs in gameplay.

4.2 Normal Forms Applied

First Normal Form (1NF)

All tables contain only atomic values. No field stores lists or repeating groups. For example, the Card table stores exactly one value per cell: name, type, base_ap_cost, base_hp, base_damage, range_attack, duration, and effect_description.

Second Normal Form (2NF)

All non-key attributes depend fully on the primary key. Tables with composite-like structures, such as Run_Hand (run_id, horde_id, card_id), were verified so that every attribute (is_retained) depends on the full key and not a subset of it.

Third Normal Form (3NF)

Transitive dependencies were identified and removed. Enemy stats (hp, damage, speed) are stored in the Enemy table rather than duplicated inside Run or Horde. Card stats are stored once in Card and referenced by foreign key everywhere else.

4.3 Justified Exception to 3NF

The Level table includes a boss_name column, which technically creates a transitive dependency: the boss name could be derived by querying the Enemy table (WHERE level_id = X AND is_boss = TRUE). The team decided to keep this column for three practical reasons:

- It simplifies queries during gameplay by avoiding an extra JOIN on every level load.
- It improves schema readability for team members.
- Boss names never change at runtime, so there is no risk of data inconsistency from this denormalization.

5. Views

Ten views are defined, split between player-facing dashboards and admin analytics. All views are computed from normalized base tables.

View Name	Audience	Purpose
v_player_overview	Player	Dashboard: lifetime stats (runs, kills, upgrades, gold, levels completed).
v_player_run_history	Player	Last 10 runs with duration, level reached, and result.
v_player_upgrades	Player	All card upgrades for a player with bonus deltas and gold spent.
v_player_best_run	Player	Deepest level, highest gold run, total kills and turns across all runs.
v_global_leaderboard	Player / Admin	Top players ranked by levels completed then enemies killed (RANK() window function).
v_death_distribution	Admin	Deaths grouped by level and horde — primary difficulty chart.
v_most_upgraded_cards	Admin	Cards ranked by purchase count, total gold, average and max tier — meta and balance tracking.
v_card_usage_frequency	Admin	Cards ranked by times dealt and retained; includes retention rate percentage.
v_horde_difficulty	Admin	Completion rate, avg turns, and avg kills per horde — balancing reference.
v_average_run_duration	Admin	Avg, min, and max run duration grouped by result (victory / defeat / in_progress).

6. Triggers

Six triggers automate denormalized aggregates in Stats and enforce business rules, avoiding manual synchronization in application code.

Trigger Name	Description
trg_after_player_insert	AFTER INSERT on Player. Auto-creates the corresponding Stats row so v_player_overview is always populated.
trg_after_run_insert	AFTER INSERT on Run. Increments Stats.total_runs for the owning player.
trg_after_run_horde_completed	AFTER UPDATE on Run_Horde. When completed flips to TRUE, increments Stats.total_enemies_killed and awards 10 gold to both Stats and Run.
trg_after_run_level_advance	AFTER UPDATE on Run. Increments Stats.levels_completed when current_level_id increases, or when result changes to 'victory'.
trg_before_card_upgrade_insert	BEFORE INSERT on Card_Upgrade. Raises SQLSTATE 45000 if level > 3, enforcing the max upgrade tier cap.
trg_after_card_upgrade_insert	AFTER INSERT on Card_Upgrade. Increments Stats.total_upgrades for the owning player.

7. Stored Procedures

Six stored procedures encapsulate all write operations. Application code calls only these procedures, never issuing raw INSERTs or UPDATEs to core tables directly.

Procedure Name	Description
sp_register_player	Inserts a new Player row. The Stats row is auto-created via trg_after_player_insert. Returns the new player_id.
sp_start_run	Inserts a new Run for a player at level 1, horde 1, result in_progress. Returns the new run_id.
sp_complete_horde	Upserts a Run_Horde record (inserts if missing, then updates). Setting completed = TRUE fires trg_after_run_horde_completed.
sp_record_death	Sets Run.result = 'defeat', is_active = FALSE, and ended_at = NOW(). Feeds v_death_distribution.
sp_upgrade_card	Inserts a Card_Upgrade row at the next tier with computed bonuses (AP cost reduction at tier 2+, HP and damage scaled by tier). Blocked by trg_before_card_upgrade_insert if tier > 3.
sp_get_player_dashboard	Returns four result sets: v_player_overview, v_player_best_run, v_player_upgrades, and the last 10 rows of v_player_run_history.

8. Contribution of Generative AI

We used Claude as a complementary tool during the database design process. The AI contributed in the following ways:

Schema structure: The AI recommended separating the run progression data into two tables, Run_Hand (cards dealt per horde) and Run_Horde (turns survived, enemies killed per horde) instead of storing all run data in a single table. This avoided repeating groups and kept each table focused on one responsibility.

Foreign key ordering: The AI flagged that Run_Hand and Card_Upgrade must be dropped before Card when resetting the schema, due to foreign key constraints. This helped the team avoid errors during development.

It is important to note that the AI didn't replace the developers decision making. All final design choices were made by us, based on the project requirements and concepts learned in class.